

Java EE — Module 6

Hibernate Framework

Detailed Study Notes

Q1. What is Hibernate? Explain its features.

Hibernate is an open-source **Object-Relational Mapping (ORM)** framework for Java. It maps Java objects (POJOs) to relational database tables, allowing developers to interact with the database using Java objects instead of writing SQL queries manually. Hibernate is the most widely used ORM framework in the Java ecosystem and implements the **JPA (Java Persistence API)** standard.

What problem does Hibernate solve?

In plain JDBC, developers must:

- Write SQL for every operation (INSERT, SELECT, UPDATE, DELETE).
- Manually map `ResultSet` columns to Java object fields.
- Handle connections, transactions, and exceptions explicitly.

Hibernate eliminates all this boilerplate by automatically handling the mapping between Java classes and database tables.

Features of Hibernate:

- **ORM (Object-Relational Mapping)** — Maps Java classes to database tables and Java fields to table columns automatically. No manual `ResultSet` parsing needed.
- **Database independence** — Hibernate uses a **dialect** system. Switching from MySQL to Oracle requires only a config change, not rewriting queries. Supports MySQL, Oracle, PostgreSQL, SQL Server, H2, and more.
- **HQL (Hibernate Query Language)** — An object-oriented query language similar to SQL but operates on Java objects, not table names. Queries are database-independent.
- **Automatic table generation** — Hibernate can automatically create, update, or validate database tables based on your Java class definitions (`hbm2ddl.auto`).
- **Caching** — Two levels of caching:
 - **First-level cache** (Session cache) — enabled by default; caches objects within a session.
 - **Second-level cache** — optional; shared across sessions using providers like EHCACHE.
- **Lazy loading** — Associated objects are loaded from the database only when accessed, not when the parent object is loaded. Reduces unnecessary DB queries.
- **Transaction management** — Integrates with JDBC transactions and JTA. Provides roll-back on failure automatically.

- **Association mapping** — Supports all relationship types: One-to-One, One-to-Many, Many-to-One, and Many-to-Many between Java classes.
- **Criteria API** — A type-safe, programmatic query API that avoids string-based queries entirely (useful for dynamic queries).
- **JPA compliance** — Hibernate implements the JPA standard, so code written with JPA annotations is portable across ORM frameworks.

Analogy: If JDBC is driving a manual car (you control every gear shift), Hibernate is driving an automatic car (the framework handles the mechanics, you just steer).

Q2. What are the advantages of ORM over JDBC?

ORM (Object-Relational Mapping) is a technique that maps the state of Java objects directly to database tables. **JDBC (Java Database Connectivity)** is the low-level API that requires developers to write and manage all SQL manually. ORM frameworks like Hibernate are built on top of JDBC but provide a much higher-level abstraction.

Side-by-side code comparison — inserting a Student:

With plain JDBC:

```

1 Connection con = DriverManager.getConnection(url, user, pass);
2 PreparedStatement ps = con.prepareStatement(
3     "INSERT INTO students (name, marks) VALUES (?, ?)");
4 ps.setString(1, "Alice");
5 ps.setDouble(2, 92.5);
6 ps.executeUpdate();
7 ps.close();
8 con.close();

```

With Hibernate (ORM):

```

1 Session session = sessionFactory.openSession();
2 session.beginTransaction();
3 Student s = new Student("Alice", 92.5);
4 session.save(s); // Hibernate generates INSERT SQL
5 session.getTransaction().commit();
6 session.close();

```

Advantages of ORM over JDBC:

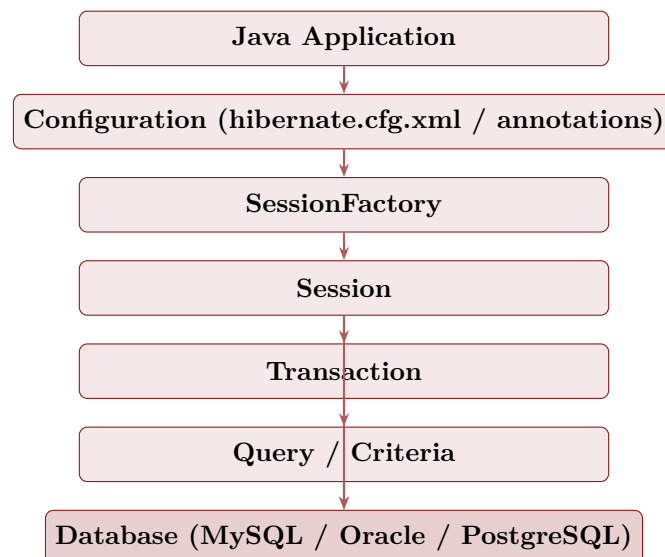
Aspect	JDBC	ORM (Hibernate)
SQL writing	Developer writes all SQL manually	SQL auto-generated from object operations
ResultSet mapping	Manual column-to-field mapping	Automatic; object populated directly
Database portability	SQL is DB-specific; rewrite for each DB	Change dialect in config; code unchanged
Boilerplate code	High (open, query, map, close)	Minimal; framework handles lifecycle
Caching	No built-in caching	First and second level caching built in
Relationship handling	Manual JOINS required	Associations mapped automatically
Transaction management	Manual try/catch/commit/rollback	Managed by framework; auto-rollback on error
Schema generation	Tables must be created manually	Hibernate can auto-create/update tables
Learning curve	Low (just SQL + Java)	Higher (must learn ORM concepts)
Fine-grained control	Full control over every query	Less control; may generate sub-optimal SQL

When to still use JDBC: For performance-critical applications requiring fine-tuned SQL (complex reporting, bulk operations), JDBC or native queries within Hibernate may be preferred. ORM shines in standard CRUD-heavy business applications.

Q3. Explain Hibernate architecture and core components.

Hibernate has a **layered architecture**. At the top is your Java application; at the bottom is the database. Hibernate sits in the middle, managing the translation between Java objects and relational data through several core components.

Architecture diagram:



Core components explained:

1. Configuration

The `Configuration` class reads the Hibernate configuration file (`hibernate.cfg.xml`) or annotations. It collects all settings: database URL, driver, dialect, mapping files, and schema generation strategy.

```
1 Configuration cfg = new Configuration();
2 cfg.configure("hibernate.cfg.xml"); // loads the config file
```

2. SessionFactory

A **heavyweight, thread-safe** object created once per application (like a connection pool). It is built from the `Configuration` and used to open `Session` objects. Creating it is expensive — create only once and reuse.

```
1 SessionFactory sessionFactory = cfg.buildSessionFactory();
```

3. Session

A **lightweight, single-threaded** object representing one unit of work with the database. It wraps a single JDBC connection. Used to save, update, delete, and query objects. Not thread-safe — create a new one per request/thread.

```
1 Session session = sessionFactory.openSession();
```

Key `Session` methods: `save()`, `get()`, `load()`, `update()`, `delete()`, `createQuery()`.

4. Transaction

Represents a database transaction. Every write operation (save, update, delete) must happen inside a transaction.

```
1 Transaction tx = session.beginTransaction();
2 // ... operations ...
3 tx.commit(); // or tx.rollback() on error
```

5. Query / Criteria

Used to execute HQL or native SQL queries. `Query` takes an HQL string; `Criteria` builds type-safe queries programmatically.

```
1 Query<Student> q = session.createQuery(
2 "FROM Student WHERE marks > :minMarks", Student.class);
3 q.setParameter("minMarks", 80.0);
4 List<Student> list = q.list();
```

6. Mapping Metadata

Tells Hibernate how Java fields map to database columns. Provided either via **XML mapping files** (`*.hbm.xml`) or **annotations** (`@Entity`, `@Table`, `@Column`, etc.).

Component	Role	Analogy
Configuration	Reads settings	Application settings file
SessionFactory	Creates Sessions	Connection pool manager
Session	One DB interaction	Single DB connection
Transaction	Groups operations	Database transaction
Query	Runs HQL/SQL	Statement/PreparedStatement
Mapping	Java ↔ DB schema	Schema blueprint

Q4. Explain the steps to set up Hibernate in a Java application.

Setting up Hibernate involves adding the required libraries, creating the configuration file, defining entity classes, and writing code to open a session and perform operations.

Step 1 — Add Hibernate dependencies (Maven pom.xml):

```

1 <dependency>
2 <groupId>org.hibernate</groupId>
3 <artifactId>hibernate-core</artifactId>
4 <version>5.6.15.Final</version>
5 </dependency>
6 <dependency>
7 <groupId>mysql</groupId>
8 <artifactId>mysql-connector-java</artifactId>
9 <version>8.0.33</version>
10 </dependency>

```

Step 2 — Create the database table:

```

1 CREATE TABLE students (
2   id    INT PRIMARY KEY AUTO_INCREMENT,
3   name  VARCHAR(100),
4   marks DOUBLE
5 );

```

Step 3 — Create the Hibernate configuration file (hibernate.cfg.xml in src/):

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE hibernate-configuration PUBLIC
3   "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4   "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5 <hibernate-configuration>
6 <session-factory>
7   <property name="hibernate.connection.driver_class">
8     com.mysql.cj.jdbc.Driver
9   </property>
10  <property name="hibernate.connection.url">
11    jdbc:mysql://localhost:3306/mydb
12  </property>
13  <property name="hibernate.connection.username">root</property>
14  <property name="hibernate.connection.password">1234</property>
15  <property name="hibernate.dialect">
16    org.hibernate.dialect.MySQL8Dialect
17  </property>
18  <property name="hibernate.hbm2ddl.auto">update</property>
19  <property name="hibernate.show_sql">true</property>

```

```

20 <mapping class="com.example.Student"/>
21 </session-factory>
22 </hibernate-configuration>

```

Step 4 — Create the Entity class:

```

1  package com.example;
2
3  import javax.persistence.*;
4
5  @Entity
6  @Table(name = "students")
7  public class Student {
8
9      @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private int id;
12
13     @Column(name = "name")
14     private String name;
15
16     @Column(name = "marks")
17     private double marks;
18
19     // No-arg constructor (required by Hibernate)
20     public Student() {}
21
22     public Student(String name, double marks) {
23         this.name = name;
24         this.marks = marks;
25     }
26
27     // Getters and setters
28     public int getId() { return id; }
29     public String getName() { return name; }
30     public void setName(String n) { this.name = n; }
31     public double getMarks() { return marks; }
32     public void setMarks(double m) { this.marks = m; }
33 }

```

Step 5 — Create the HibernateUtil helper class:

```

1  package com.example;
2
3  import org.hibernate.SessionFactory;
4  import org.hibernate.cfg.Configuration;
5
6  public class HibernateUtil {
7
8      private static final SessionFactory sessionFactory;
9
10     static {
11         try {
12             sessionFactory = new Configuration()
13                 .configure() // reads hibernate.cfg.xml
14                 .buildSessionFactory();
15         } catch (Exception e) {
16             throw new ExceptionInInitializerError(e);
17         }
18     }
19 }

```

```

17     }
18 }
19
20 public static SessionFactory getSessionFactory() {
21     return sessionFactory;
22 }
23
24 public static void shutdown() {
25     sessionFactory.close();
26 }
27 }

```

Step 6 — Test with a simple save operation:

```

1 import org.hibernate.Session;
2 import org.hibernate.Transaction;
3
4 public class Main {
5     public static void main(String[] args) {
6         Session session = HibernateUtil.getSessionFactory().openSession();
7         Transaction tx = session.beginTransaction();
8
9         Student s = new Student("Alice", 92.5);
10        session.save(s);
11
12        tx.commit();
13        session.close();
14        System.out.println("Student saved with ID: " + s.getId());
15        HibernateUtil.shutdown();
16    }
17 }

```

hbm2ddl.auto values: **create** — drops and recreates tables every run. **update** — alters tables to match entities without dropping data. **validate** — checks schema matches entities; throws error if not. **none** — does nothing to the schema. Use **update** during development, **validate** or **none** in production.

Q5. What is mapping? How do you map Java classes to database tables?

Mapping in Hibernate is the process of defining a correspondence between a **Java class** and a **database table**, and between **Java fields** and **table columns**. Hibernate uses this mapping to automatically generate SQL for all CRUD operations.

Mapping can be done in two ways: using **XML mapping files** (.hbm.xml) or using **JPA annotations** directly on the Java class.

What gets mapped:

Java concept	Database concept
Class	Table
Object (instance)	Row
Field / Property	Column
Primary key field	Primary key column
Reference (association)	Foreign key
Collection field	Join table or child table rows

Method 1 — XML Mapping file (Student.hbm.xml):

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE hibernate-mapping PUBLIC
3 "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4 "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5 <hibernate-mapping>
6 <class name="com.example.Student" table="students">
7
8 <id name="id" column="id">
9 <generator class="identity"/>    <!-- AUTO_INCREMENT -->
10 </id>
11
12 <property name="name" column="name" type="string"/>
13 <property name="marks" column="marks" type="double"/>
14
15 </class>
16 </hibernate-mapping>

```

Register this file in hibernate.cfg.xml with:

```

1 <mapping resource="com/example/Student.hbm.xml"/>

```

Method 2 — Annotation-based mapping (preferred, modern):

```

1 import javax.persistence.*;
2
3 @Entity                                // marks this class as a Hibernate entity
4 @Table(name = "students")              // maps to the "students" table
5 public class Student {
6
7     @Id                                // primary key
8     @GeneratedValue(strategy = GenerationType.IDENTITY) // AUTO_INCREMENT
9     @Column(name = "id")
10    private int id;
11
12    @Column(name = "name", length = 100, nullable = false)
13    private String name;
14
15    @Column(name = "marks")
16    private double marks;
17
18    // no-arg constructor, getters, setters ...
19 }

```

Mapping associations:

Many-to-One (Student belongs to a Department):


```

1  @Entity
2  @Table(name = "students")
3  public class Student {
4      @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5      private int id;
6
7      private String name;
8
9      @ManyToOne                                // many students to one department
10     @JoinColumn(name = "dept_id")             // foreign key column in students table
11     private Department department;
12 }

```

One-to-Many (Department has many Students):

```

1  @Entity
2  @Table(name = "departments")
3  public class Department {
4      @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5      private int id;
6
7      private String deptName;
8
9      @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
10     private List<Student> students = new ArrayList<>();
11 }

```

Q6. Explain Hibernate configuration using XML.

The **Hibernate XML configuration** uses a file named `hibernate.cfg.xml` placed in the classpath root (`src/` or `src/main/resources/`). It specifies all the settings Hibernate needs to connect to the database, choose the correct SQL dialect, manage the schema, and locate entity mappings.

Complete annotated hibernate.cfg.xml:

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE hibernate-configuration PUBLIC
3  "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4  "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5
6  <hibernate-configuration>
7  <session-factory>
8
9  <!-- ===== JDBC Connection Settings ===== -->
10 <property name="hibernate.connection.driver_class">
11 com.mysql.cj.jdbc.Driver
12 </property>
13 <property name="hibernate.connection.url">
14 jdbc:mysql://localhost:3306/mydb?useSSL=false
15 </property>
16 <property name="hibernate.connection.username">root</property>
17 <property name="hibernate.connection.password">password</property>
18

```

```

19 <!-- ===== Connection Pool Size ===== -->
20 <property name="hibernate.connection.pool_size">10</property>
21
22 <!-- ===== SQL Dialect ===== -->
23 <!-- Tells Hibernate which SQL variant to generate -->
24 <property name="hibernate.dialect">
25 org.hibernate.dialect.MySQL8Dialect
26 </property>
27
28 <!-- ===== Schema Management ===== -->
29 <!-- create / create-drop / update / validate / none -->
30 <property name="hibernate.hbm2ddl.auto">update</property>
31
32 <!-- ===== Logging ===== -->
33 <!-- Print generated SQL to console (dev only) -->
34 <property name="hibernate.show_sql">true</property>
35 <!-- Format the printed SQL for readability -->
36 <property name="hibernate.format_sql">true</property>
37
38 <!-- ===== Second-level Cache (optional) ===== -->
39 <property name="hibernate.cache.use_second_level_cache">
40 false
41 </property>
42
43 <!-- ===== Entity Mappings ===== -->
44 <!-- Option A: XML mapping files -->
45 <mapping resource="com/example/Student.hbm.xml"/>
46
47 <!-- Option B: Annotated classes -->
48 <mapping class="com.example.Student"/>
49 <mapping class="com.example.Department"/>
50
51 </session-factory>
52 </hibernate-configuration>

```

Loading the configuration in Java:

```

1 // Reads hibernate.cfg.xml automatically from classpath
2 SessionFactory sf = new Configuration()
3 .configure()
4 .buildSessionFactory();

```

Important property reference:

Property	Purpose
connection.driver_class	JDBC driver class name
connection.url	JDBC connection URL
connection.username	Database username
connection.password	Database password
dialect	SQL dialect for the target database
hbm2ddl.auto	Schema generation strategy
show_sql	Print generated SQL to console
format_sql	Format the printed SQL
connection.pool_size	Built-in connection pool size

Q7. Explain Hibernate configuration using annotations.

Instead of a separate XML mapping file, Hibernate supports **JPA annotations** placed directly on the Java entity class. This is the **modern, preferred approach** as it keeps the mapping metadata close to the code. The `hibernate.cfg.xml` is still used for connection settings, but `<mapping class="...">` replaces `<mapping resource="...">`.

Core JPA / Hibernate annotations:

Annotation	Purpose
@Entity	Marks class as a Hibernate-managed entity
@Table(name="...")	Specifies the database table name
@Id	Marks the primary key field
@GeneratedValue	Auto-generation strategy for the primary key
@Column(name="...")	Maps field to a specific column name
@Transient	Field is NOT persisted to the database
@Temporal	Maps <code>java.util.Date</code> to SQL date type
@Lob	Maps large object (CLOB / BLOB) fields
@ManyToOne	Many-to-one association
@OneToMany	One-to-many association
@OneToOne	One-to-one association
@ManyToMany	Many-to-many association
@JoinColumn	Specifies the foreign key column

Complete annotated entity example:

```
1 package com.example;
2
3 import javax.persistence.*;
4 import java.util.Date;
5
6 @Entity
7 @Table(name = "employees")
8 public class Employee {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     @Column(name = "emp_id")
13     private int empId;
14
15     @Column(name = "first_name", length = 50, nullable = false)
16     private String firstName;
17
18     @Column(name = "last_name", length = 50)
19     private String lastName;
20
21     @Column(name = "salary")
22     private double salary;
23
24     @Temporal(TemporalType.DATE)    // maps to SQL DATE
25     @Column(name = "join_date")
26     private Date joinDate;
27
28     @Transient                      // not stored in database
```

```

29     private String tempToken;
30
31     @Lob                                // for large text fields
32     @Column(name = "bio")
33     private String bio;
34
35     // No-arg constructor (mandatory for Hibernate)
36     public Employee() {}
37
38     // Getters and setters ...
39 }

```

Register in hibernate.cfg.xml:

```

1  <!-- No hbm.xml files needed; just reference the class -->
2  <mapping class="com.example.Employee"/>
3  <mapping class="com.example.Department"/>

```

Or use addAnnotatedClass() in Java:

```

1  SessionFactory sf = new Configuration()
2  .configure()
3  .addAnnotatedClass(Employee.class)
4  .addAnnotatedClass(Department.class)
5  .buildSessionFactory();

```

XML vs Annotations: Use **annotations** for most projects — they are concise and co-located with the class. Use **XML mappings** when you need to keep Java classes free of framework dependencies (e.g., reusing the same class in non-JPA contexts).

Q8. Explain basic CRUD operations in Hibernate.

CRUD stands for **Create, Read, Update, Delete** — the four fundamental database operations. In Hibernate, these are performed using **Session** methods instead of SQL statements. The **Session** object is the primary interface for all persistence operations.

Setup (shared across all examples):

```

1  SessionFactory sf = HibernateUtil.getSessionFactory();

```

CREATE — Save a new record:

```

1  Session session = sf.openSession();
2  Transaction tx = session.beginTransaction();
3
4  Student s = new Student();
5  s.setName("Alice");
6  s.setMarks(92.5);
7
8  session.save(s); // Hibernate generates: INSERT INTO students ...
9  // s.getId() is populated after save
10
11 tx.commit();
12 session.close();

```

```
13 System.out.println("Saved with ID: " + s.getId());
```

READ — Retrieve a record by primary key:

```
1 Session session = sf.openSession();
2
3 // get() returns null if not found
4 Student s = session.get(Student.class, 1);
5
6 // load() throws ObjectNotFoundException if not found (lazy)
7 // Student s = session.load(Student.class, 1);
8
9 if (s != null) {
10     System.out.println("Name: " + s.getName());
11     System.out.println("Marks: " + s.getMarks());
12 }
13 session.close();
```

READ — Retrieve all records using HQL:

```
1 Session session = sf.openSession();
2
3 List<Student> students = session
4 .createQuery("FROM Student", Student.class)
5 .list();
6
7 for (Student s : students) {
8     System.out.println(s.getId() + " - " + s.getName());
9 }
10 session.close();
```

UPDATE — Modify an existing record:

```
1 Session session = sf.openSession();
2 Transaction tx = session.beginTransaction();
3
4 // First fetch the object
5 Student s = session.get(Student.class, 1);
6
7 if (s != null) {
8     s.setMarks(95.0);           // modify the field
9     session.update(s);         // Hibernate generates: UPDATE students ...
10 }
11
12 tx.commit();
13 session.close();
```

DELETE — Remove a record:

```
1 Session session = sf.openSession();
2 Transaction tx = session.beginTransaction();
3
4 Student s = session.get(Student.class, 1);
5
6 if (s != null) {
7     session.delete(s);         // Hibernate generates: DELETE FROM students ...
8 }
```

```

8 }
9
10 tx.commit();
11 session.close();

```

Using `saveOrUpdate()` — create if new, update if exists:

```

1 Session session = sf.openSession();
2 Transaction tx = session.beginTransaction();
3
4 Student s = new Student();
5 s.setId(1); // if ID 1 exists -> UPDATE, else -> INSERT
6 s.setName("Bob");
7 s.setMarks(88.0);
8
9 session.saveOrUpdate(s);
10
11 tx.commit();
12 session.close();
13

```

Session method summary:

Operation	Method	SQL generated
Create	<code>session.save(obj)</code>	INSERT INTO ...
Read	<code>session.get(Class, id)</code>	SELECT * FROM ... WHERE id=?
Update	<code>session.update(obj)</code>	UPDATE ... SET ... WHERE id=?
Delete	<code>session.delete(obj)</code>	DELETE FROM ... WHERE id=?
Both	<code>session.saveOrUpdate(obj)</code>	INSERT or UPDATE

Q9. Explain the basics of Hibernate Query Language (HQL).

HQL (Hibernate Query Language) is an object-oriented query language provided by Hibernate. It is syntactically similar to SQL but instead of operating on **table names and column names**, it operates on **Java class names and field names**. Hibernate translates HQL into the appropriate SQL for the target database at runtime.

Key characteristics of HQL:

- Case-insensitive for keywords (FROM, WHERE, ORDER BY).
- Case-sensitive for Java class names and field names.
- Uses Java class name, not table name: FROM `Student` not FROM `students`.
- Uses Java field name, not column name: WHERE `s.marks` not WHERE `marks_col`.
- Supports SELECT, WHERE, ORDER BY, GROUP BY, JOIN, aggregate functions.
- Supports named parameters (`:paramName`) and positional parameters (`?`).

1. Select all objects:

```

1 // "Student" is the Java class name, NOT the table name
2 List<Student> list = session

```

```

3 .createQuery("FROM Student", Student.class)
4 .list();
5

```

2. Select with WHERE clause:

```

1 // Named parameter (preferred)
2 List<Student> list = session
3 .createQuery("FROM Student WHERE marks > :min", Student.class)
4 .setParameter("min", 75.0)
5 .list();
6

```

3. Select specific fields (returns Object array):

```

1 List<Object[]> rows = session
2 .createQuery("SELECT s.name, s.marks FROM Student s")
3 .list();
4
5 for (Object[] row : rows) {
6     System.out.println(row[0] + " -> " + row[1]);
7 }
8

```

4. ORDER BY:

```

1 List<Student> list = session
2 .createQuery("FROM Student ORDER BY marks DESC", Student.class)
3 .list();
4

```

5. Aggregate functions:

```

1 // COUNT
2 Long count = (Long) session
3 .createQuery("SELECT COUNT(s) FROM Student s")
4 .uniqueResult();
5
6 // AVG, MAX, MIN, SUM
7 Double avg = (Double) session
8 .createQuery("SELECT AVG(s.marks) FROM Student s")
9 .uniqueResult();
10
11 System.out.println("Total: " + count + ", Avg: " + avg);

```

6. UPDATE and DELETE in HQL (bulk operations):

```

1 // Bulk update (no need to load objects into memory)
2 Transaction tx = session.beginTransaction();
3
4 int updated = session
5 .createQuery("UPDATE Student SET marks = marks + 5
6 WHERE marks < 50")
7 .executeUpdate();
8
9 int deleted = session
10 .createQuery("DELETE FROM Student WHERE marks < 35")
11 .executeUpdate();

```

```

12
13 tx.commit();
14 System.out.println("Updated: " + updated + ", Deleted: " + deleted);

```

7. JOIN (HQL with associations):

```

1 // Fetch students with their department (assumes @ManyToOne mapping)
2 List<Student> list = session
3 .createQuery(
4 "FROM Student s JOIN FETCH s.department WHERE
5 s.department.deptName = :dept",
6 Student.class)
7 .setParameter("dept", "Computer Science")
8 .list();

```

8. Pagination:

```

1 List<Student> page = session
2 .createQuery("FROM Student ORDER BY name", Student.class)
3 .setFirstResult(0) // offset (0-based)
4 .setMaxResults(10) // page size
5 .list();

```

Q10. Differentiate between SQL and HQL.

SQL (Structured Query Language) is the standard language for interacting with relational databases. It operates on **tables and columns**. **HQL (Hibernate Query Language)** is Hibernate's query language that operates on **Java classes and fields**. HQL is translated into SQL by Hibernate at runtime.

Side-by-side query comparison:

SQL	HQL
SELECT * FROM students	FROM Student
SELECT name, marks FROM students	SELECT s.name, s.marks FROM Student s
SELECT * FROM students WHERE marks > 80	FROM Student WHERE marks > 80
SELECT * FROM students ORDER BY marks DESC	FROM Student ORDER BY marks DESC
SELECT COUNT(*) FROM students	SELECT COUNT(s) FROM Student s
UPDATE students SET marks=90 WHERE id=1	UPDATE Student SET marks=90 WHERE id=1
DELETE FROM students WHERE id=1	DELETE FROM Student WHERE id=1
SELECT * FROM students s JOIN departments d ON s.dept_id=d.id	FROM Student s JOIN FETCH s.department

Detailed comparison table:

Feature	SQL	HQL
Operates on	Database tables and columns	Java classes and fields
Database dependency	Tied to a specific DB (MySQL, Oracle)	Database-independent; Hibernate translates
Object awareness	No concept of objects	Fully object-oriented; supports inheritance
Joins	Manual JOIN with ON condition	JOIN FETCH via object associations
Result type	Raw rows (need manual mapping)	Directly returns Java objects
Polymorphism	Not supported	Supports polymorphic queries on hierarchies
Portability	Not portable across databases	Fully portable; Hibernate handles dialect
Table/column names	Must use exact DB names	Uses Java class/field names
NULL handling	IS NULL, IS NOT NULL	Same syntax supported
Native queries	Yes (this is native SQL)	Can fall back to SQL via <code>createNativeQuery()</code>

When to use native SQL in Hibernate: For complex queries that HQL cannot express (e.g., database-specific functions, complex subqueries, stored procedure calls), Hibernate allows native SQL via `session.createNativeQuery("SELECT ...")`. This escapes the ORM abstraction when needed.